Imam Abdulrahman Bin Faisal University
College of Computer Science and Information Technology

ARTI 404 — Natural Language Processing

Lab 4 — Classification and Evaluation

Student Name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1

Classification and Evaluation in NLP

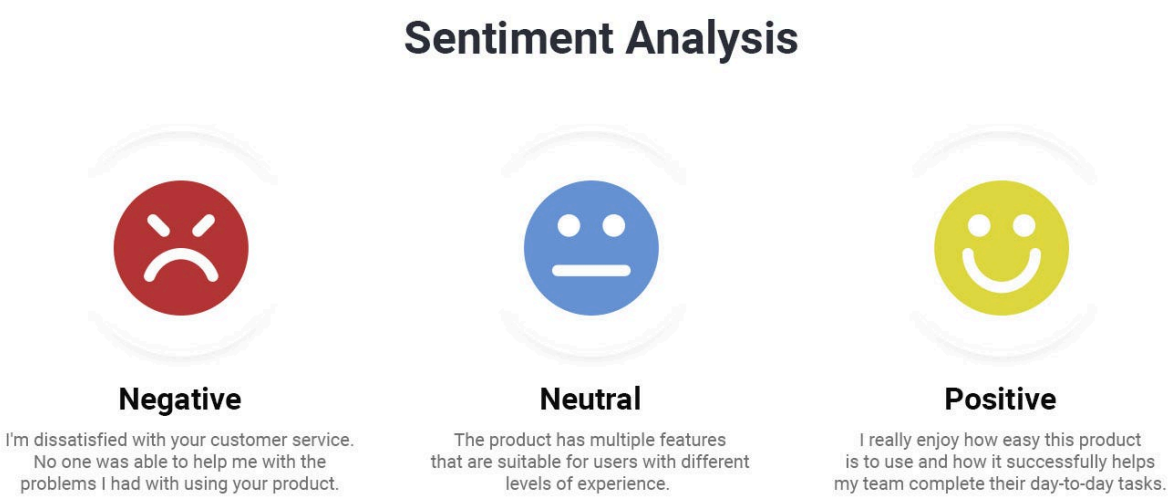
Text classification is a common Natural Language Processing (NLP) task where a model assigns a predefined category or label to a piece of text.

Examples include:
- Spam detection
- Sentiment analysis
- News categorization

In this lab, we will learn how to build and evaluate a text classification model.

Case Study: Sentiment Analysis

Sentiment analysis, also known as opinion mining, its goal is to understand the information present in the text and categorize it as positive, negative, or neutral.



Disneyland Reviews Dataset

In this lab we'll look into Disneyland Reviews dataset.
Link: https://www.kaggle.com/datasets/arushchillar/disneyland-reviews

The dataset includes 42,000 reviews of 3 Disneyland branches - Paris, California and Hong Kong, posted by visitors on Trip Advisor.

Column Description:

- Review_ID: unique id given to each review
- Rating: ranging from 1 (unsatisfied) to 5 (satisfied)
- Year_Month: when the reviewer visited the theme park
- Reviewer_Location: country of origin of visitor
- Review_Text: comments made by visitor

- Disneyland_Branch: location of Disneyland Park

```python
# Load the dataset
import pandas as pd, kagglehub, os, glob

dl_path = kagglehub.dataset_download("arushchillar/disneyland-reviews")
csv_file = glob.glob(os.path.join(dl_path, "*.csv"))[0]
data = pd.read_csv(csv_file, encoding='latin-1')
data
```

	Review_ID	Rating	Year_Month	Reviewer_Location	Review_Text	Branch
0	670772142	4	2019-4	Australia	If you've ever been to Disneyland anywhere you...	Disneyland_HongKong
1	670682799	4	2019-5	Philippines	Its been a while since d last time we visit HK...	Disneyland_HongKong
2	670623270	4	2019-4	United Arab Emirates	Thanks God it wasn t too hot or too humid wh...	Disneyland_HongKong
3	670607911	4	2019-4	Australia	HK Disneyland is a great compact park. Unfortu...	Disneyland_HongKong
4	670607296	4	2019-4	United Kingdom	the location is not in the city, took around 1...	Disneyland_HongKong
...	...	...	...	...	...	...
42651	1765031	5	missing	United Kingdom	i went to disneyland paris in july 03 and thou...	Disneyland_Paris
42652	1659553	5	missing	Canada	2 adults and 1 child of 11 visited Disneyland ...	Disneyland_Paris
42653	1645894	5	missing	South Africa	My eleven year old daughter and myself went to...	Disneyland_Paris
42654	1618637	4	missing	United States	This hotel, part of the Disneyland Paris compl...	Disneyland_Paris
42655	1536786	4	missing	United Kingdom	I went to the Disneyparis resort, in 1996, wit...	Disneyland_Paris

42656 rows × 6 columns

```
#Check for missing values
data.isna().sum()
```

```
Review_ID           0
Rating              0
Year_Month          0
Reviewer_Location   0
Review_Text         0
Branch              0
dtype: int64
```

```
# Drop rows with missing values in the relevant columns
data = data.dropna(subset=['Review_Text', 'Rating'])
```

```
# Map star ratings to sentiments
data['Sentiment'] = data['Rating'].apply(lambda rating: 'positive' if rating > 3 else
('negative' if rating < 3 else 'neutral'))
data.head()
```

	Review_ID	Rating	Year_Month	Reviewer_Location	Review_Text	Branch	Sentiment
0	670772142	4	2019-4	Australia	If you've ever been to Disneyland anywhere you...	Disneyland_HongKong	positive
1	670682799	4	2019-5	Philippines	Its been a while since d last time we visit HK...	Disneyland_HongKong	positive
2	670623270	4	2019-4	United Arab Emirates	Thanks God it wasn t too hot or too humid wh...	Disneyland_HongKong	positive
3	670607911	4	2019-4	Australia	HK Disneyland is a great compact park. Unfortu...	Disneyland_HongKong	positive
4	670607296	4	2019-4	United Kingdom	the location is not in the city, took around 1...	Disneyland_HongKong	positive

```
# Import necessary libraries
import re
import nltk

def preprocess_text(text):

    # 1. Convert text to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-z\s]', '', text)

    # 3. Tokenize the text
    words = text.split()

    return ' '.join(words)

# Apply preprocessing to the review text
data['Review_Text'] = data['Review_Text'].apply(preprocess_text)

# Display cleaned reviews
data[['Review_Text', 'Sentiment']].head()
```

	Review_Text	Sentiment
0	if youve ever been to disneyland anywhere youl...	positive
1	its been a while since d last time we visit hk...	positive
2	thanks god it wasn t too hot or too humid when...	positive
3	hk disneyland is a great compact park unfortun...	positive
4	the location is not in the city took around ho...	positive

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Split the dataset into training and testing sets
train_data, test_data, train_labels, test_labels = train_test_split(data['Review_Text'],
data['Sentiment'], test_size=0.2, random_state=42)


# TF-IDF Vectorization
tfidf_vectorizer = TfidfVectorizer(max_features=1000)  # You can adjust max_features
based on your dataset size - limit vocabulary to the 1000 most informative tokens
train_vectors = tfidf_vectorizer.fit_transform(train_data)
test_vectors = tfidf_vectorizer.transform(test_data)
```

```
print(f"train_data size: {train_data.shape}")
print(f"test_data size: {test_data.shape}")
```

```
train_data size: (34124,)
test_data size: (8532,)
```

```python
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Train the Support Vector Machine (SVM) classifier
svm_classifier = SVC(kernel='linear')
svm_classifier.fit(train_vectors, train_labels)

# Predictions on the test set
predictions = svm_classifier.predict(test_vectors)

# Evaluate the model
accuracy = accuracy_score(test_labels, predictions)
print(f"Accuracy: {accuracy:.2f}")

# Display classification report
print("Classification Report:")
print(classification_report(test_labels, predictions))
```

```
Accuracy: 0.84
Classification Report:
              precision    recall  f1-score   support

    negative       0.64      0.54      0.58       720
     neutral       0.48      0.18      0.26      1035
    positive       0.88      0.98      0.92      6777

    accuracy                           0.84      8532
   macro avg       0.66      0.56      0.59      8532
weighted avg       0.81      0.84      0.81      8532
```

Use Case: Amazon reviews of unlocked phone analysis

The objective of this use case is to conduct sentiment analysis on Amazon reviews of unlocked phones, categorizing reviews into three classes: positive, negative, and neutral. The goal is to gain a comprehensive understanding of customer opinions and sentiments regarding various unlocked phone models.

- Dataset link: https://www.kaggle.com/datasets/PromptCloudHQ/amazon-reviews-unlocked-mobile-phones

Task1: Load the data and do 5 different preprocessing steps on it

Task2: Use a proper train and test split

Task3: Do Feature Extraction Using TF-IDF

Task4: Train a Naive Bayes Classifier

Task5: Evaluation the model on the Test Set

Task6: Print the Confusion Matrix

```python
# Task 1: Load the data
import kagglehub, os, glob, re
import pandas as pd

amz_path = kagglehub.dataset_download("PromptCloudHQ/amazon-reviews-unlocked-mobile-phones")
print("Path:", amz_path)
print("Files:", os.listdir(amz_path))

csv_file = glob.glob(os.path.join(amz_path, "*.csv"))[0]
amazon = pd.read_csv(csv_file)

print("\nShape:", amazon.shape)
print("Columns:", list(amazon.columns))
amazon.head()
```

```
Downloading to C:\Users\ahmad\.cache\kagglehub\datasets\PromptCloudHQ\amazon-reviews-unlocked-mobile-phones\1.archive...
```

```
100%|██████████| 32.6M/32.6M [00:03<00:00, 9.80MB/s]
```

```
Extracting files...
```

```
Path: C:\Users\ahmad\.cache\kagglehub\datasets\PromptCloudHQ\amazon-reviews-unlocked-mobile-phones\versions\1
Files: ['Amazon_Unlocked_Mobile.csv']

Shape: (413840, 6)
Columns: ['Product Name', 'Brand Name', 'Price', 'Rating', 'Reviews', 'Review Votes']
```

	Product Name	Brand Name	Price	Rating	Reviews	Review Votes
0	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	5	I feel so LUCKY to have found this used (phone...	1.0
1	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	nice phone, nice up grade from my pantach revu...	0.0
2	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	5	Very pleased	0.0
3	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	It works good but it goes slow sometimes but i...	0.0
4	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	Great phone to replace my lost phone. The only...	0.0

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download('stopwords')
nltk.download('wordnet')

# Keep the needed columns and drop missing values
amazon = amazon[['Reviews', 'Rating']].dropna()

# Map star ratings to sentiment labels
amazon['Sentiment'] = amazon['Rating'].apply(
    lambda r: 'positive' if r > 3 else ('negative' if r < 3 else 'neutral'))

# Work on a random sample to keep the training time reasonable
amazon = amazon.sample(n=50000, random_state=42).reset_index(drop=True)

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    text = str(text).lower()                          # 1. Lowercasing
    text = re.sub(r'http\S+|www\.\S+', ' ', text)   # 2. Remove URLs
    text = re.sub(r'[^a-z\s]', ' ', text)           # 3. Remove punctuation, digits,
special chars
    words = text.split()                              # 4. Tokenization
    words = [w for w in words if w not in stop_words and len(w) > 2]  # 5. Stop-word
removal
    words = [lemmatizer.lemmatize(w) for w in words]                 # 6. Lemmatization
    return ' '.join(words)

amazon['Clean_Review'] = amazon['Reviews'].apply(preprocess_text)
amazon = amazon[amazon['Clean_Review'].str.split().str.len() >=
2].reset_index(drop=True)

print("Reviews after preprocessing:", len(amazon))
print("\nBefore:", amazon['Reviews'][0][:120])
print("After :", amazon['Clean_Review'][0][:120])
amazon[['Reviews', 'Clean_Review', 'Rating', 'Sentiment']].head()
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\ahmad\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\ahmad\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
Reviews after preprocessing: 43507

Before: I needed a texting phone that did not require a data plan. I also needed a Nokia as
they have the best antenna and can g
After : needed texting phone require data plan also needed nokia best antenna get cell phone
buyer beware phone difficult use lo
```

	Reviews	Clean_Review	Rating	Sentiment
0	I needed a texting phone that did not require ...	needed texting phone require data plan also ne...	2	negative
1	I recommend this excellent team for the many d...	recommend excellent team many differentiating ...	5	positive
2	updated review. Purchased this phone	updated review purchased phone	5	positive

	Reviews	Clean_Review	Rating	Sentiment
	on Novemb...	november worked...		
3	battery fail	battery fail	3	neutral
4	The phone was orange pink color not yellow lik...	phone orange pink color yellow like ordered	1	negative

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    amazon['Clean_Review'], amazon['Sentiment'],
    test_size=0.2, random_state=42, stratify=amazon['Sentiment'])

print("Training size:", X_train.shape[0])
print("Testing size :", X_test.shape[0])
print("\nClass distribution in the training set:")
print(y_train.value_counts())
```

```
Training size: 34805
Testing size : 8702

Class distribution in the training set:
Sentiment
positive    22853
negative     9107
neutral      2845
Name: count, dtype: int64
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
train_vectors = tfidf_vectorizer.fit_transform(X_train)
test_vectors = tfidf_vectorizer.transform(X_test)

print("Training matrix shape:", train_vectors.shape)
print("Testing matrix shape :", test_vectors.shape)
print("\nExample features:", tfidf_vectorizer.get_feature_names_out()[:20])
```

```
Training matrix shape: (34805, 5000)
Testing matrix shape : (8702, 5000)

Example features: ['ability' 'able' 'able activate' 'able connect' 'able download'
 'able find' 'able get' 'able make' 'able send' 'able use' 'abroad'
 'absolute' 'absolutely' 'absolutely love' 'abuse' 'accept' 'acceptable'
 'accepted' 'accesories' 'access']
```

```
from sklearn.naive_bayes import MultinomialNB

nb_classifier = MultinomialNB()
nb_classifier.fit(train_vectors, y_train)

predictions = nb_classifier.predict(test_vectors)
print("Naive Bayes classifier trained successfully.")
print("First 10 predictions:", predictions[:10])
```

```
Naive Bayes classifier trained successfully.
First 10 predictions: ['negative' 'positive' 'positive' 'positive' 'positive' 'positive'
 'negative' 'positive' 'positive' 'positive']
```

```python
from sklearn.metrics import accuracy_score, classification_report

accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, predictions))
```

```
Accuracy: 0.8312

Classification Report:
              precision    recall  f1-score   support

    negative       0.80      0.76      0.78      2277
     neutral       0.33      0.03      0.05       711
    positive       0.85      0.96      0.90      5714

    accuracy                           0.83      8702
   macro avg       0.66      0.58      0.58      8702
weighted avg       0.79      0.83      0.80      8702
```

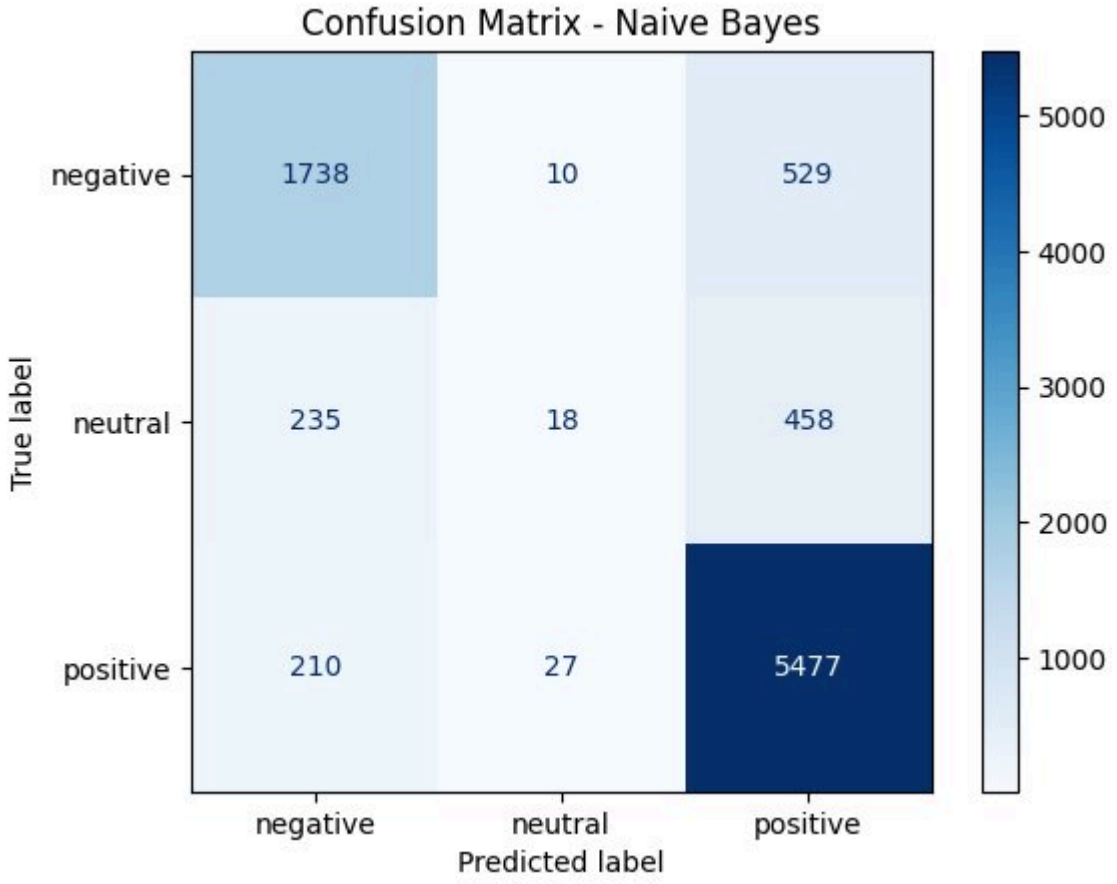
```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

labels = ['negative', 'neutral', 'positive']
cm = confusion_matrix(y_test, predictions, labels=labels)

print("Confusion Matrix:")
print(pd.DataFrame(cm,
                   index=[f"actual {l}" for l in labels],
                   columns=[f"predicted {l}" for l in labels]))

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Naive Bayes")
plt.show()
```

```
Confusion Matrix:
                 predicted negative  predicted neutral  predicted positive
actual negative                1738                 10                 529
actual neutral                  235                 18                 458
actual positive                 210                 27                5477
```



Summary of the Use Case: Amazon Reviews of Unlocked Phones

Task 1 — Loading and pre-processing. The dataset *Amazon Reviews: Unlocked Mobile Phones* (Kaggle, *PromptCloudHQ/amazon-reviews-unlocked-mobile-phones*) was downloaded with `kagglehub`. It contains 413,840 reviews in 6 columns; only `Reviews` and `Rating` were kept. Star ratings were mapped to sentiment labels using the same rule as the Disneyland example (rating > 3 → positive, rating < 3 → negative, rating = 3 → neutral), and a random

sample of 50,000 reviews was taken to keep the training time reasonable. Six pre-processing steps were applied (one more than the five required):

1. Lower casing
2. Removing URLs
3. Removing punctuation, digits and special characters
4. Tokenization
5. Stop-word removal (NLTK English stop words, plus tokens shorter than 3 characters)
6. Lemmatization with `WordNetLemmatizer`

Reviews left with fewer than two words were dropped, leaving **43,507** reviews.

Task 2 — Train/test split. An 80/20 split with `random_state=42` and `stratify` on the label, so that the three classes keep the same proportions in both sets: **34,805** training reviews and **8,702** test reviews. The training set is clearly imbalanced: 22,853 positive, 9,107 negative and only 2,845 neutral.

Task 3 — Feature extraction. `TfidfVectorizer` with `max_features=5000` and `ngram_range=(1, 2)`, so that both single words and two-word phrases (for example *absolutely love*, *able connect*) are used as features. This produced a 34,805 × 5,000 training matrix and an 8,702 × 5,000 test matrix.

Task 4 — Classifier. A `MultinomialNB` (Multinomial Naive Bayes) classifier, which is the standard Naive Bayes variant for word-count and TF-IDF features.

Task 5 — Evaluation. Overall accuracy on the test set: **0.8312**.

Class	Precision	Recall	F1-score	Support
negative	0.80	0.76	0.78	2,277
neutral	0.33	0.03	0.05	711
positive	0.85	0.96	0.90	5,714
accuracy	**0.83**			8,702
macro avg	0.66	0.58	0.58	8,702
weighted avg	0.79	0.83	0.80	8,702

Task 6 — Confusion matrix and discussion. The matrix shows that the model handles the positive class very well (5,477 of 5,714 correct) and the negative class reasonably well (1,738 of 2,277 correct), but almost completely fails on the neutral class: only 18 of 711 neutral reviews were identified, while 458 of them were predicted as positive and 235 as negative.

There are two reasons for this. First, the classes are strongly imbalanced — neutral reviews are only about 8% of the training data, so the prior probability of that class is low and Naive Bayes rarely selects it. Second, a 3-star review is genuinely mixed in wording: it contains the same vocabulary found in positive and negative reviews, so the words themselves carry little signal for separating it. The same pattern appears in the Disneyland example earlier in this lab, where the SVM classifier also reached a high overall accuracy (0.84) but a recall of only 0.18 on the neutral class. This is why accuracy alone is misleading on imbalanced data, and

why the per-class report and the confusion matrix are necessary: the macro average F1-score (0.58) reflects the real performance of the model far better than the accuracy of 0.83.